

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM126

Name: Sanskar Gorave

Division: B

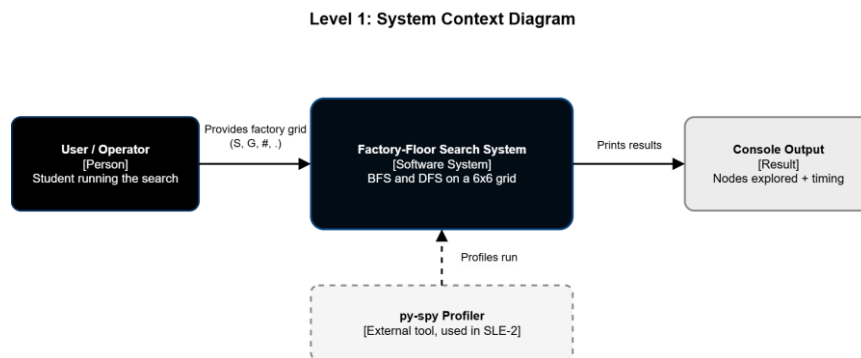
Date: 04/10/2026

GitHub / Portfolio Link (optional): <https://github.com/SanskarG-09/SLE--AI.git>

1. System Title & Short Description

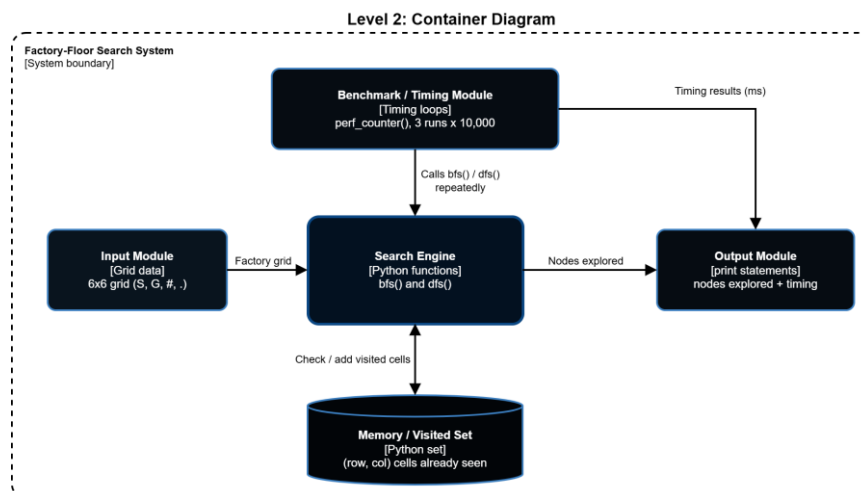
Factory-Floor Search System (BFS vs DFS). This system models a 6×6 textile factory floor as a grid, where S is the start, G is the Quality Inspection Station, # is an obstacle and . is free floor. It searches the grid with Breadth-First Search (BFS) and Depth-First Search (DFS) to reach the goal. For each algorithm it reports the number of nodes explored, and it measures running time over 10,000 repeated searches. The problem it solves is comparing the two uninformed search strategies for finding a route to a target on a factory layout. In SLE-2, BFS explored 22 nodes and DFS explored 17, and the timing results (per 10,000 searches) were 226.39 ms for BFS and 141.38 ms for DFS.

2. Context Diagram (Level 1)



The Factory-Floor Search System is the single system being designed. The user (the student running the program) supplies the 6×6 factory grid with S, G, # and . symbols. The system runs BFS and DFS on it and prints the nodes explored and the timing to the console. The py-spy profiler sits outside the system and is used only to profile its runs, as in SLE-2.

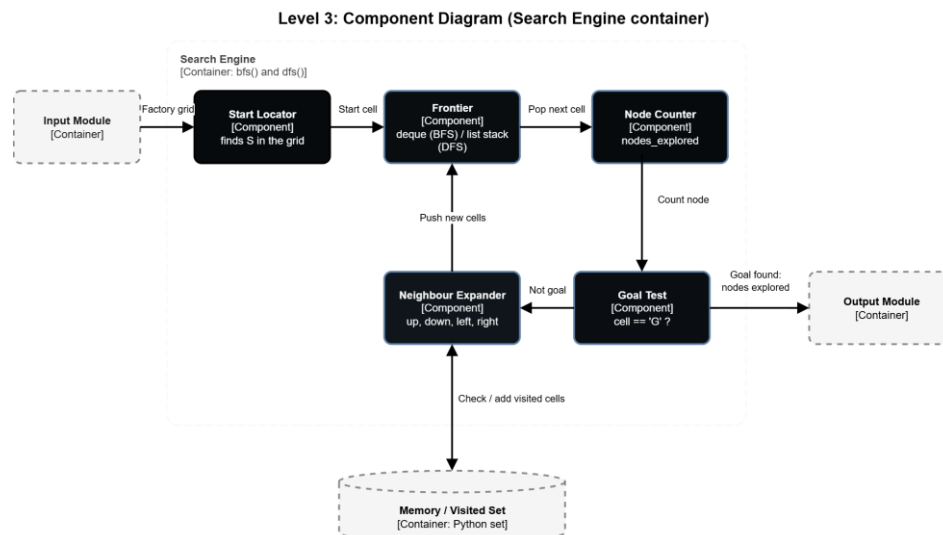
3. Container Diagram (Level 2)



- **Input Module:** holds the 6×6 factory grid (S, G, #, .), hardcoded in the file, and gives it to the search functions.
- **Search Engine:** bfs() and dfs() find S, expand the four neighbours (up, down, left, right), stop at G, and return the number of nodes explored.

- **Memory / Visited Set:** a Python set of (row, col) cells already queued, so no cell is searched twice and loops are avoided.
- **Benchmark / Timing Module:** loops that call each search 10,000 times, 3 runs each, timed with `perf_counter()`.
- **Output Module:** the print statements that show the nodes explored and the timing results.

4. Component Diagram (Level 3)



The Search Engine (`bfs()` and `dfs()`) has five components. The Start Locator finds S in the grid. The Frontier holds the cells waiting to be explored: a deque in BFS and a list used as a stack in DFS. Each cell taken from the Frontier is counted by the Node Counter and checked by the Goal Test. If it is not G, the Neighbour Expander tries up, down, left and right, skips walls, out-of-bounds cells and visited cells, and pushes the new cells onto the Frontier. The Frontier is the only real difference between BFS and DFS.

5. Code Level Overview (Level 4)

- **factory:** the 6x6 grid (a list of lists with S, G, #, .) that both searches read.
- **def bfs(factory):** breadth-first search using a deque as a FIFO queue; returns the nodes explored.
- **def dfs(factory):** depth-first search using a list as a LIFO stack; returns the nodes explored.
- **BFS timing loop:** runs `bfs()` 10,000 times per run, 3 runs, timed with `time.perf_counter()`.
- **DFS timing loop:** the same for `dfs()`.

6. Design Decisions

I split the system into five parts (input, search engine, visited set, benchmark and output) so each part has one clear job and the search can be timed without changing its logic. BFS and DFS use the same code structure and differ only in the frontier, a deque for BFS and a stack for DFS, so the comparison between them is fair. A visited set stops cells from being searched twice, which avoids loops on the grid. I kept the grid hardcoded and the output as a node count, because the goal was comparing the searches, not building a full path-finder.

7. AI Contribution Note

- **AI tools used:** ChatGPT and Claude.
- **What AI helped with:** Providing the code in `factory_search.py` (BFS and DFS on the factory grid, with timing), explaining the SLE-3 guideline step by step, suggesting the C4 structure for the system, creating the diagrams as draw.io files, and drafting the text of the report sections.
- **What I did myself:** Chose my SLE-2 BFS vs DFS factory-floor system as the base for SLE-3, ran `factory_search.py` in VS Code, opened the diagrams in draw.io, and exported them as PNG images.

8. Conclusion

Drawing the same BFS vs DFS system at four C4 levels showed me that architecture means looking at one system at different levels of detail, from the big picture down to the code. The Context level showed what the system talks to, and the Container level split it into five parts with separate jobs. The Component level showed that BFS and DFS differ only in the Frontier, and the Code level tied each part back to real functions. This made it clearer

how the search, the visited set and the timing code fit together, and where a change such as adding A* with a heuristic would go.